Malware Analysis

Monnappa K A, Learning Malware Analysis, Packt Publishing Ltd, 2018..

Shubha Malige

Asst.Professor
CSE(Cyber Security)

Unit II

Inspecting PE Header Information, Comparing and Classifying The malware. **Dynamic Analysis:** System And Network Monitoring, Dynamic Analysis (Monitoring) Tools, Dynamic Analysis Steps, Putting it All Together: Analyzing a Malware Executable, Dynamic Link Library (DLL) Analysis.

- Pedagogy: Chalk and board, Problem based learning, Demonstration.

6. Inspecting PE Header Information

Inspecting PE Header Information

What is a PE (Portable Executable) file?

Standard format for Windows executable files (.exe, .dll, .sys, etc.)

Contains the program's code and data, ready for Windows to load and run

Why is the PE header important?

Acts like a "table of contents" for the executable

Provides critical info: where the program starts, libraries used, and resources included

Role in Malware Analysis:

- ✔ Inspecting the PE header helps identify suspicious or malicious files
- ✔ Reveals hidden or unusual dependencies and behaviors in malware
- ✔ Essential first step before executing or analyzing a file

Inspecting PE Header Information

PE format: Windows executable files (.exe, .dll, .sys, etc.) follow the Portable Executable (PE) format. It is a standard structure that Windows uses to understand and run programs.

A PE file contains a header (PE header) and other sections. The header stores important details like:

- Where the program should be loaded in memory
- Where execution should start (entry point)
- Required libraries (DLLs)
- Resources used by the program

When you run a program, the Windows loader reads the PE header, loads the program into memory based on that information, and then starts executing it.

File Dependencies and Imports in Malware

Malware interacts with the operating system to perform actions like reading or writing into files, modifying the registry, or communicating over the network. Windows provides these services through APIs (Application Programming Interfaces), which are functions stored in DLL (Dynamic Link Library) files.

Executables, including malware, use these functions by importing them from the DLLs. The collection of imported functions is stored in the **Import Table** of the PE file. By inspecting these imports, analysts can gain insights into what the malware is capable of doing, without even running it. This forms a crucial part of **static malware analysis**.

For example, if a malware executable wants to create a file on disk, on Windows, it can use an API CreateFile(), which is exported in kernel32.dll. To call the API, it first has to load kernel32.dll into its memory and then call the CreateFile() function.

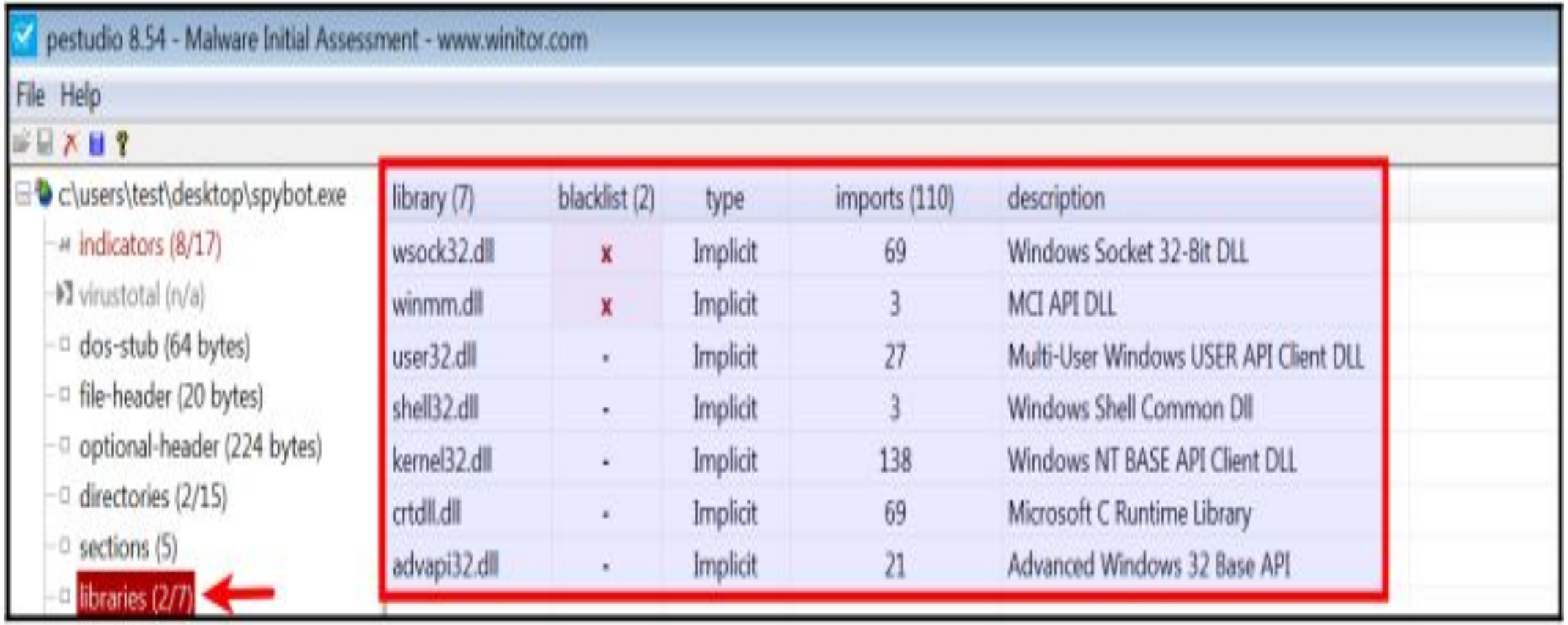
Using Tools to Inspect Imports

- ✔ Tools like pestudio can be used to examine a malware sample's dependencies and imported functions. The Libraries tab shows all the DLLs that the executable depends on, which will be loaded into memory during execution.

- ✔ The Imports tab displays all the functions the executable imports from these DLLs. For example, if the malware imports network-related functions such as connect or send from wsock32.dll, it indicates that the malware is likely to perform network operations.

- ✔ pestudio also highlights functions that are commonly used by malware in a blacklisted column, making it easier for analysts to spot suspicious behavior.

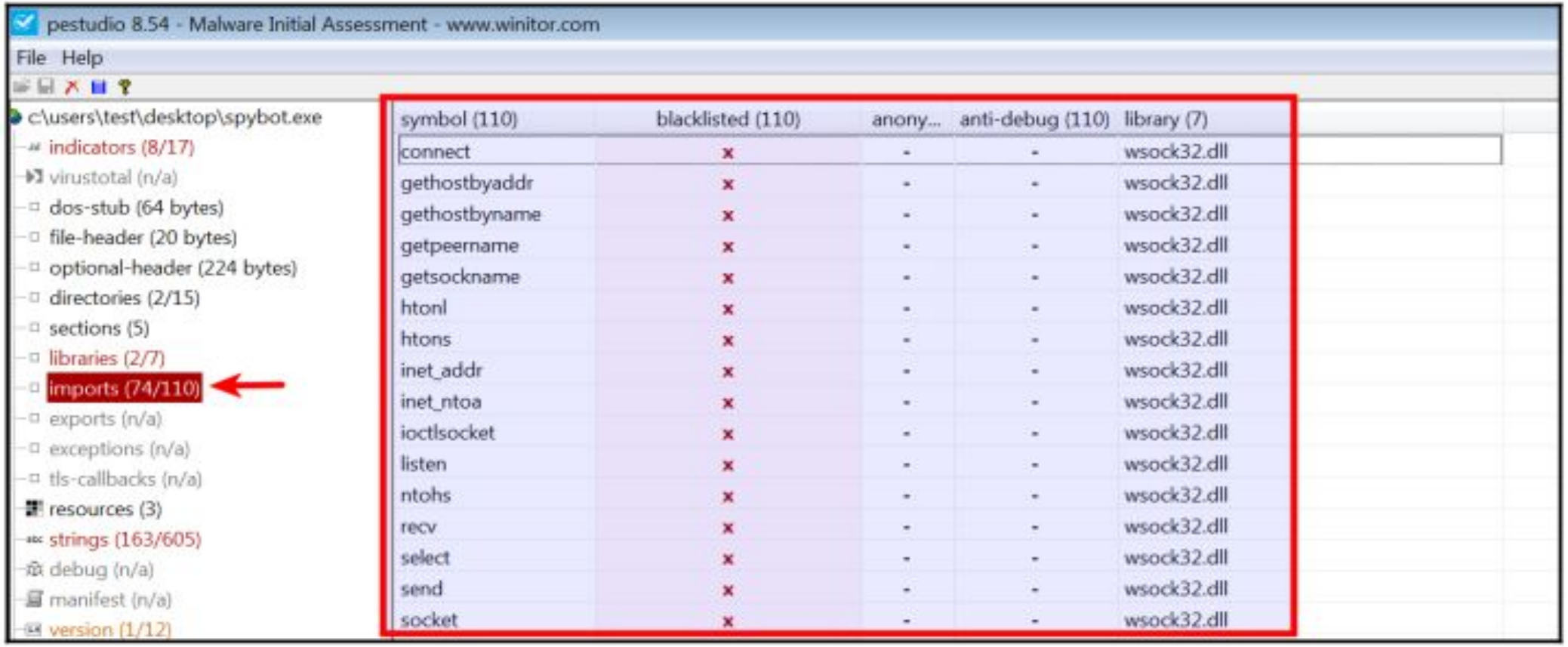
File Dependencies and Imports in Malware



library (7)	blacklist (2)	type	imports (110)	description
wsock32.dll	x	Implicit	69	Windows Socket 32-Bit DLL
winmm.dll	x	Implicit	3	MCI API DLL
user32.dll	-	Implicit	27	Multi-User Windows USER API Client DLL
shell32.dll	-	Implicit	3	Windows Shell Common Dll
kernel32.dll	-	Implicit	138	Windows NT BASE API Client DLL
crtdll.dll	-	Implicit	69	Microsoft C Runtime Library
advapi32.dll	-	Implicit	21	Advanced Windows 32 Base API

Highlight key DLLs like `kernel32.dll`, `wsock32.dll`.

File Dependencies and Imports in Malware



Highlight network functions like `connect`, `send`, etc.

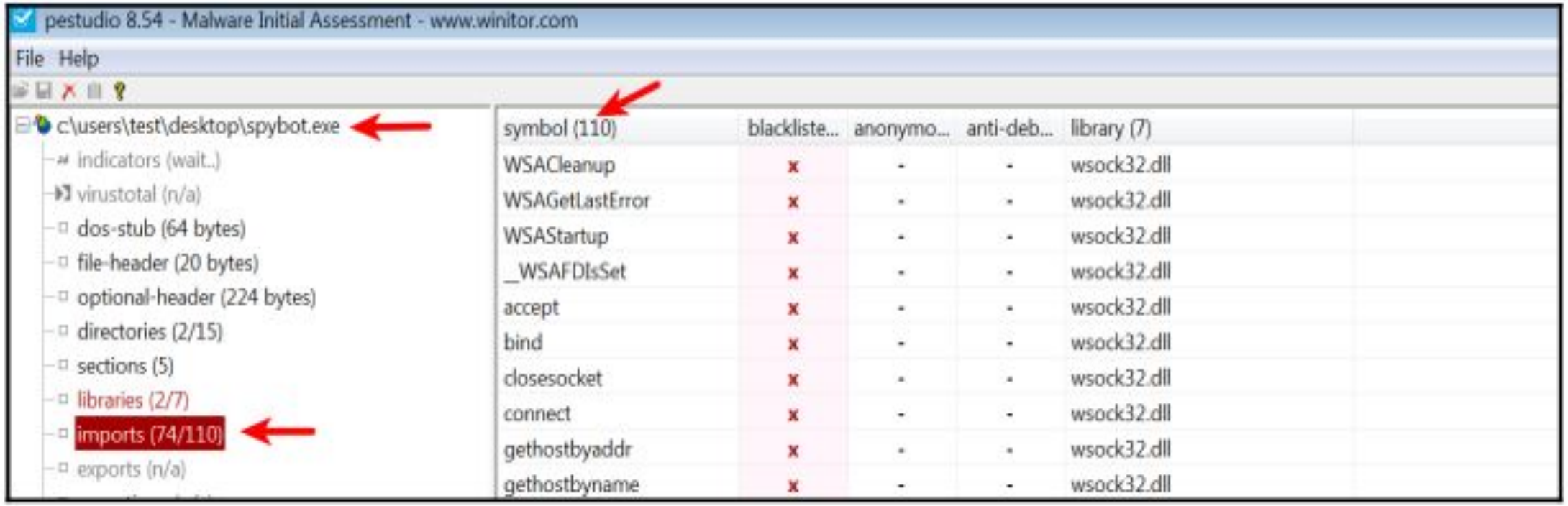
Detecting Packed or Obfuscated Malware

Malware can sometimes be packed or obfuscated to hide its functionality. Packed malware typically shows very few imports in the Import Table because the real code is compressed or encrypted. By contrast, an unpacked sample shows all the imports normally.

For example, an unpacked Spybot sample may list over 100 imports, while the packed version may show only a dozen. This difference is a strong indicator that the malware is packed, and the real functions will only be visible after unpacking.

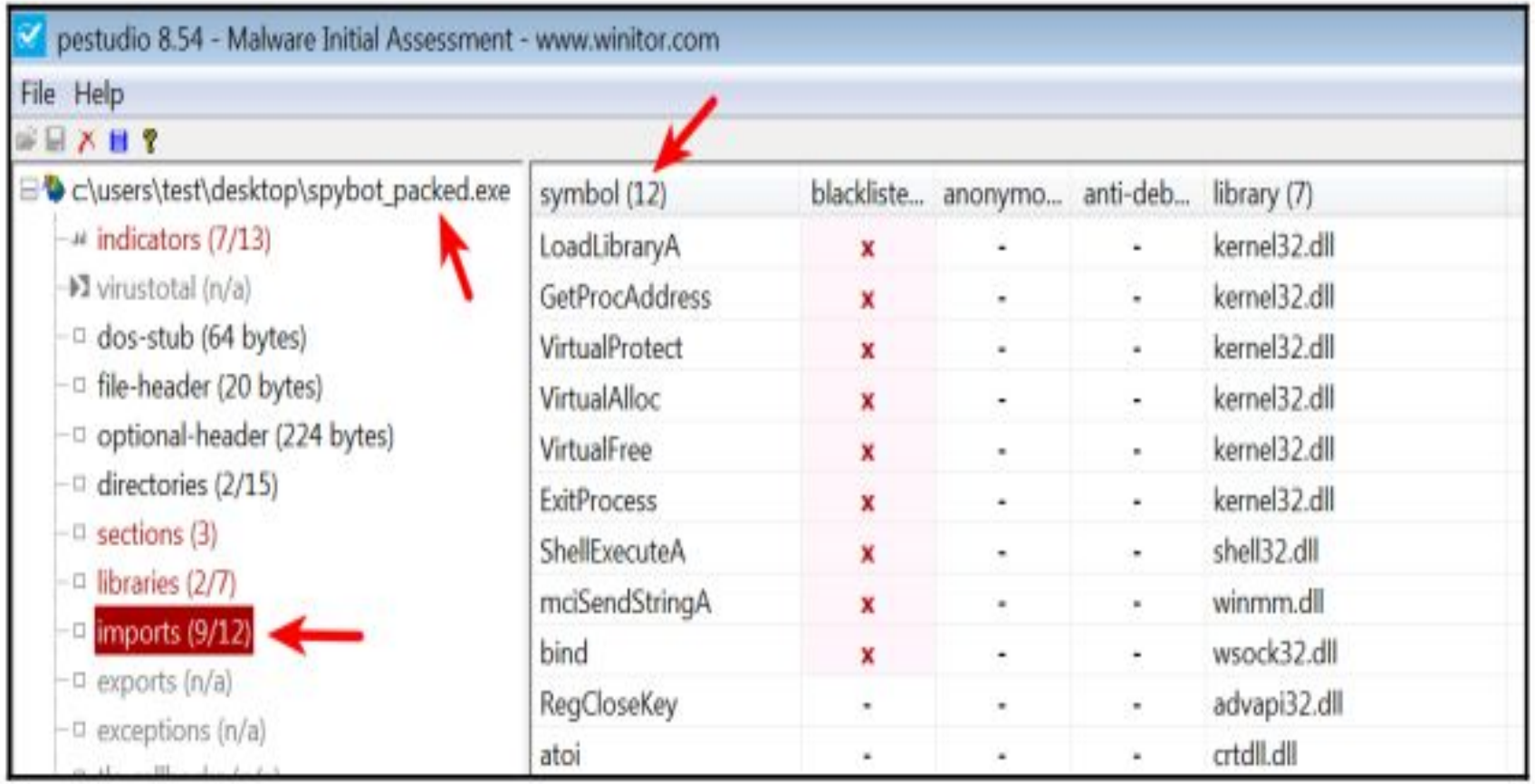
Detecting Packed or Obfuscated Malware

To demonstrate that, let's compare the imports between the *unpacked sample of spybot* and the *packed spybot sample*. The following screenshot shows 110 imports in the unpacked spybot sample:



Detecting Packed or Obfuscated Malware

On the other hand, the *packed sample* of spybot shows only 12 imports:



symbol (12)	blackliste...	anonymo...	anti-deb...	library (7)
LoadLibraryA	x	-	-	kernel32.dll
GetProcAddress	x	-	-	kernel32.dll
VirtualProtect	x	-	-	kernel32.dll
VirtualAlloc	x	-	-	kernel32.dll
VirtualFree	x	-	-	kernel32.dll
ExitProcess	x	-	-	kernel32.dll
ShellExecuteA	x	-	-	shell32.dll
mciSendStringA	x	-	-	winmm.dll
bind	x	-	-	wsock32.dll
RegCloseKey	-	-	-	advapi32.dll
atoi	-	-	-	crtdll.dll

Runtime DLL loading and Limitations

- ✔ Not all DLLs and functions appear in the Import Table. Malware can dynamically load DLLs during execution using APIs like LoadLibrary() or LdrLoadDLL() and can resolve function addresses with GetProcAddress().

- ✔ These runtime-loaded DLLs are not visible in the Import Table, so static analysis alone may not reveal all functionality. Analysts must sometimes combine static and dynamic analysis to fully understand the malware's behavior. This slide emphasizes the limitations of import table inspection.

Automating Import Analysis with Python

- ✔ For analyzing multiple malware samples, imports can be enumerated programmatically using Python's pefile module. This allows analysts to list all DLLs and imported functions from a PE file efficiently.

- ✔ A simple Python script can open a PE file, iterate through its Import Table, and print the names of the DLLs and their imported functions.

- ✔ This approach is useful when dealing with large datasets or when manually inspecting each file is impractical. The output typically lists DLLs like KERNEL32.DLL and functions such as LoadLibraryA, GetProcAddress, and VirtualAlloc, which can reveal the malware's capabilities.

Runtime DLL loading and Limitations

```
import pefile
import sys

mal_file = sys.argv[1]
pe = pefile.PE(mal_file)
if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
    for entry in pe.DIRECTORY_ENTRY_IMPORT:
        print "%s" % entry.dll
        for imp in entry.imports:
            if imp.name != None:
                print "\t%s" % (imp.name)
            else:
                print "\tord(%s)" % (str(imp.ordinal))
        print "\n"
```

Runtime DLL loading and Limitations

The following is the result of running the preceding script against the `spybot_packed.exe` sample; from the output, you can see the list of DLLs and imported functions:

```
$ python enum_imports.py spybot_packed.exe
KERNEL32.DLL
  LoadLibraryA
GetProcAddress
VirtualProtect
VirtualAlloc
VirtualFree
ExitProcess

ADVAPI32.DLL
  RegCloseKey

CRTDLL.DLL
  atoi
[...REMOVED....]
```

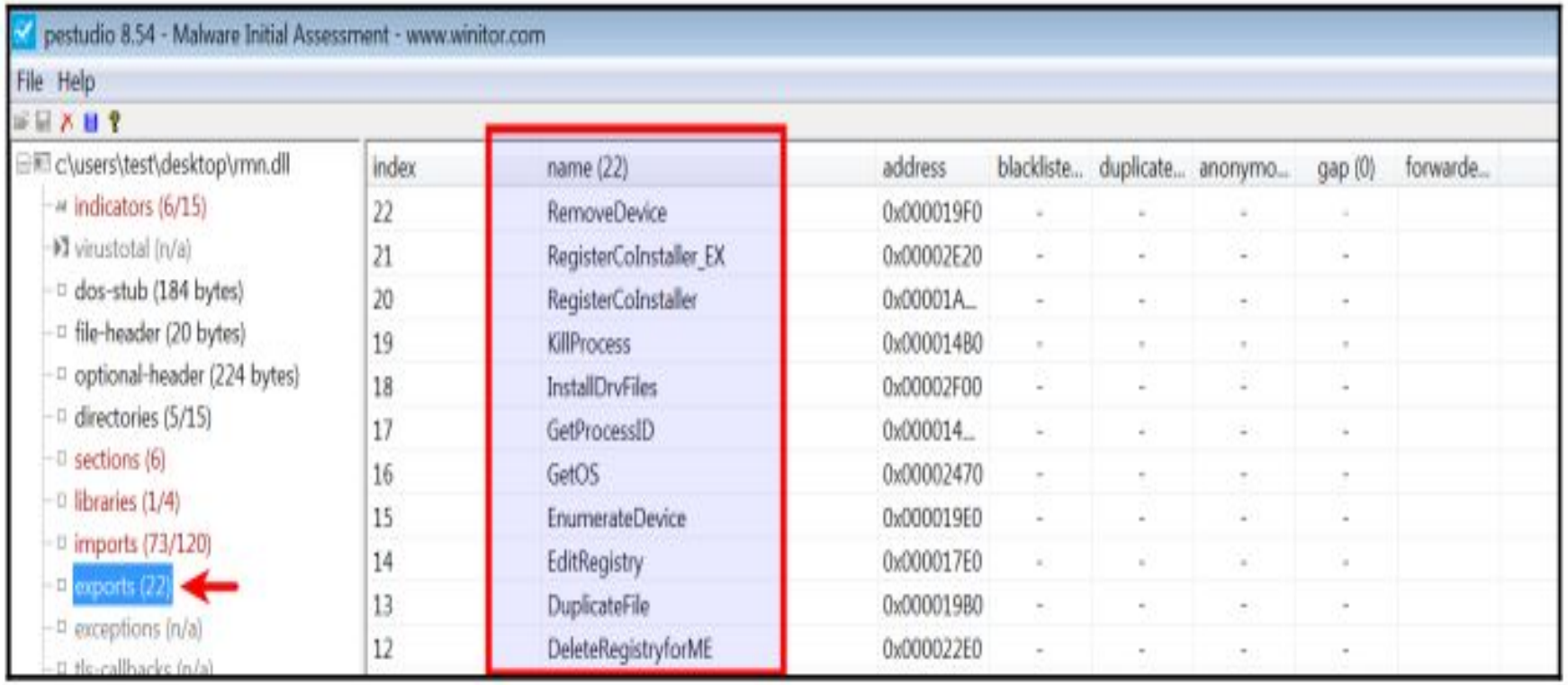
Inspecting Exports

What are DLL Exports?

- ✔ A DLL can export functions that other programs, such as executables, can call. Unlike an executable, a DLL cannot run by itself, it requires a host process to load it and execute its code.
- ✔ Malware authors often create DLLs that export malicious functions. By inspecting these exports, analysts can get a preliminary idea of the DLL's capabilities, such as which system areas it may modify or which operations it may perform.
- ✔ Inspecting exported functions helps predict what a DLL may do when loaded. For example, a malware DLL may export functions that manipulate the registry, delete files, or communicate over the network.
- ✔ However, analysts should be cautious because attackers may use misleading or random export names to hide the true purpose of the DLL. Despite this, examining exports can guide further investigation.

Tools to inspect imports

Tools like pestudio can display all exported functions in a DLL. For instance, the Ramnit malware DLL shows exports that suggest registry modifications and system manipulation. Analysts can quickly identify potentially malicious functionality from the exported function names.



Automating Export Analysis with Python

Python's pefile module can enumerate DLL exports efficiently. Using a simple script, you can list all exported functions of a DLL without opening it in a GUI tool. This is especially useful for analyzing multiple malware samples. Example output from the Ramnit DLL includes function names such as:

AddRegistryforME

CreateBridgeRegistryfor2K

CleanupDevice

CreateFolder

Automating Export Analysis with Python

In Python, the exported functions can be enumerated using the *pefile module*, as shown here:

```
$ python
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
>>> import pefile
>>> pe = pefile.PE("rmn.dll")
>>> if hasattr(pe, 'DIRECTORY_ENTRY_EXPORT'):
...     for exp in pe.DIRECTORY_ENTRY_EXPORT.symbols:
...         print "%s" % exp.name
```

Automating Export Analysis with Python

```
AddDriverPath
AddRegistryforME
CleanupDevice
CleanupDevice_EX
CreateBridgeRegistryfor2K
CreateFolder
CreateKey
CreateRegistry
DeleteDriverPath
DeleteOemFile
DeleteOemInfFile
DeleteRegistryforME
DuplicateFile
EditRegistry
EnumerateDevice
GetOS
[.....REMOVED....]
```

Examining PE Section Table and Sections

- ✔ The actual content of the PE file is divided into sections.
- ✔ The sections are immediately followed by the PE header.
- ✔ These sections represent either code or data and they have in memory attributes such as read/write.
- ✔ The section representing code contains instructions that will be executed by the processor, whereas the section containing data can represent different types of data, such as read/write program data (global variables), import/export tables, resources, and so on

Examining PE Section Table and Sections

- ✔ A PE file is divided into **sections** representing either **code or data**.
- ✔ Code sections (like .text) contain instructions executed by the CPU, while data sections (like .data, .rdata, .rsrc) contain variables, resources, or import/export tables.
- ✔ Sections have **attributes** such as read, write, or execute, which define how they behave in memory.
- ✔ During the compilation of the executable, consistent section names are added by the compilers. The following table outlines some of the common sections in a PE file:

Examining PE Section Table and Sections

Section Name	Description
.text or CODE	Contains executable code.
.data or DATA	Typically Contains read/write data and global variables.
.rdata	Contains read-only data. Sometimes it also contains import and export information.
.idata	If present, contains the import table. If not present, then the import information is stored in .rdata section.
.edata	If present, contains export information. If not present, then the export information is found in .rdata section.
.rsrc	This section contains the resources used by the executable such as icons, dialogs, menus, strings, and so on.

Examining PE Section Table and Sections

Section Names and Suspicious Sections:

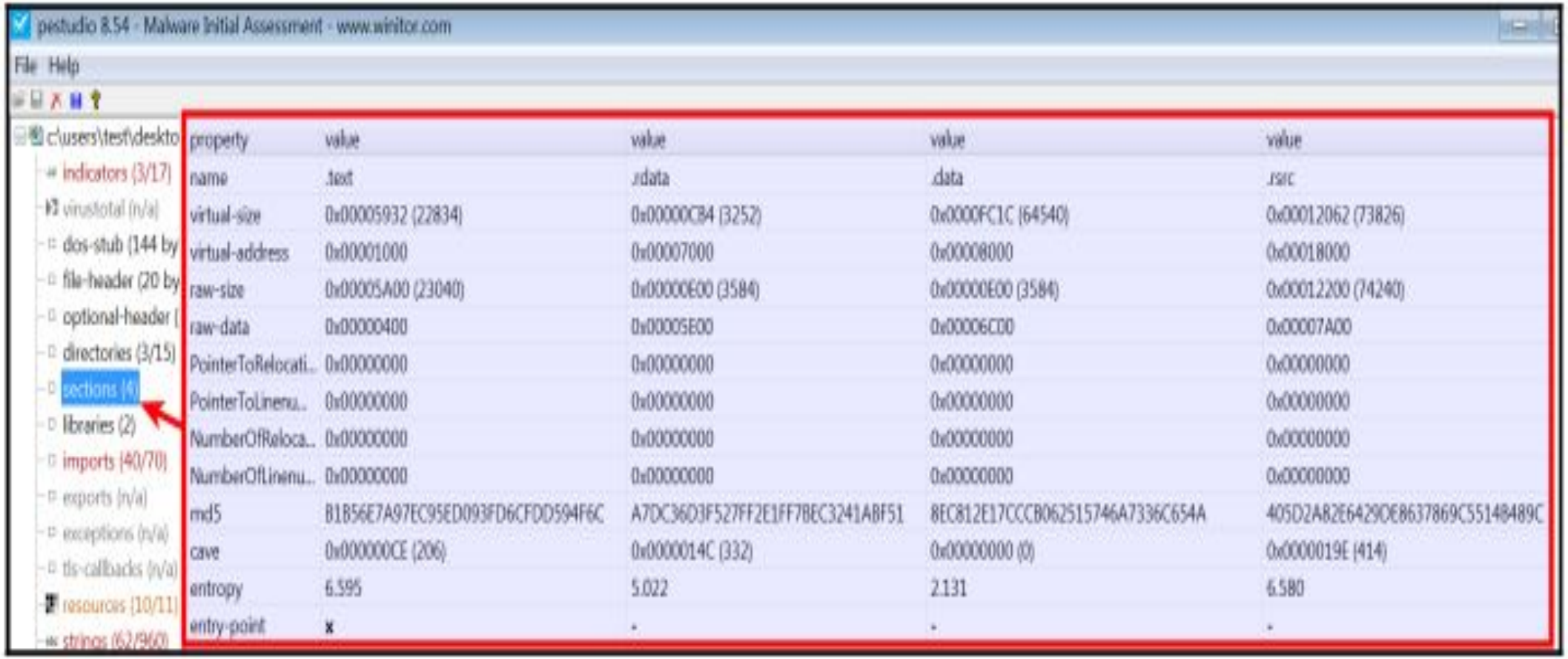
- ✔ Section names in a PE file are mainly for human reference, not enforced by the OS.
- ✔ Unusual or uncommon section names can indicate obfuscation or malicious behavior and should be analyzed further.

Section Table Information:

- ✔ The section table in the PE header contains details about each section, including its name, location, size, and attributes (read/write/execute).
- ✔ Tools like pestudio can display this information, helping analysts understand the executable's structure and detect suspicious sections.

Examining PE Section Table and Sections

When you load an executable in pestudio and click on sections, it displays the section information extracted from the section table and its attributes (read/write and so on).



property	value	value	value	value
name	.text	.rdata	.data	.rsrc
virtual-size	0x00005932 (22834)	0x00000C84 (3252)	0x0000FC1C (64540)	0x00012062 (73826)
virtual-address	0x00001000	0x00007000	0x00008000	0x00018000
raw-size	0x00005A00 (23040)	0x00000E00 (3584)	0x00000E00 (3584)	0x00012200 (74240)
raw-data	0x00000400	0x00005E00	0x00006C00	0x00007A00
PointerToRelocati...	0x00000000	0x00000000	0x00000000	0x00000000
PointerToLinenu...	0x00000000	0x00000000	0x00000000	0x00000000
NumberOfReloca...	0x00000000	0x00000000	0x00000000	0x00000000
NumberOfLinenu...	0x00000000	0x00000000	0x00000000	0x00000000
md5	B1B56E7A97EC95ED093FD6CFDD594F6C	A7DC36D3F527FF2E1FF7BEC3241ABF51	8EC812E17CCCB062515746A7336C654A	405D2A82E6429CE8637869C55148489C
cave	0x000000CE (206)	0x0000014C (332)	0x00000000 (0)	0x0000019E (414)
entropy	6.595	5.022	2.131	6.580
entry-point	x	-	-	-

Examining PE Section Table and Sections

This table explains the **important fields in the PE Section Header** of a Windows executable file.
It indicates how different sections of a PE file are organized both **on disk** and **in memory**, and where execution begins.

Field	Description
Names	Displays section names. In this case, the executable contains four sections (`.text`, `.data`, `.rdata` and `.rsrc`).
Virtual-Size	Indicates the size of the section when loaded into memory.
Virtual-Address	This is the relative virtual address (that is, offset from the base address of the executable) where the section can be found in memory.
Raw-size	Indicates the size of the section on the disk.
Raw-data	Indicates the offset in the file where the section can be found.
Entry-point	This is the RVA (relative virtual address) where the code starts executing. In this case, the entry point is in the `.text` section, which is normal.

Examining PE Section Table and Sections

✔ Unusual Section Names and Entry Point:

Packed malware often uses non-standard section names like UPX0 and UPX1 instead of common compiler sections such as .text or .data.

If the entry point lies in a section like UPX1, it indicates that execution begins in a decompression routine, which is typical for packed binaries.

✔ Raw Size vs Virtual Size Mismatch:

In normal executables, the raw size (on disk) and virtual size (in memory) of sections are nearly equal.

A section with raw size = 0 but a large virtual size is a strong sign of packing, as the malware unpacks its real code into memory during execution.

Examining PE Section Table and Sections

pestudio 8.54 - Malware Initial Assessment - www.winitor.com

File Help

c:\users\test\desktop\olib.exe

- indicators (5/19)
- virustotal (n/a)
- dos-stub (144 bytes)
- file-header (20 bytes)
- optional-header (224 bytes)
- directories (2/15)
- sections (3)
- libraries (2)
- imports (7/7)
- exports (n/a)
- exceptions (n/a)
- tls-callbacks (n/a)
- resources (10/11)
- strings (9/786)

property	value	value	value
name	UPX0	UPX1	.rsrc
virtual-size	0x0001F000 (126976)	0x0000E000 (57344)	0x00006000 (24576)
virtual-address	0x00001000	0x00020000	0x0002E000
raw-size	0x00000000 (0)	0x0000D200 (53760)	0x00006000 (24576)
raw-data	0x00000400	0x00000400	0x0000D600
PointerToRelocations	0x00000000	0x00000000	0x00000000
PointerToLinenumbers	0x00000000	0x00000000	0x00000000
NumberOfRelocations	0x00000000	0x00000000	0x00000000
NumberOfLinenumbers	0x00000000	0x00000000	0x00000000
md5	n/a	F11986D7AB6C5393A98882E653711E43	6565582E0719707A3AC6CC57D11ACD71
cave	0x00000000 (0)	0x00000000 (0)	0x00000000 (0)
entropy	n/a	7.890	5.675
entry-point	-	x	-

Examining PE Section Table and Sections

The following Python script demonstrates the use of the pefile module to display the section and its characteristics:

```
import pefile
import sys

# Load the PE file provided as a command-line argument
pe = pefile.PE(sys.argv[1])

# Iterate through all sections in the PE file
for section in pe.sections:
    print(
        "%s %s %s %s" % (
            section.Name,                    # Section name
            hex(section.VirtualAddress),        # RVA (memory
location)
            hex(section.Misc_VirtualSize),       # Size in memory
            section.SizeOfRawData               # Size on disk
        )
    )

print("\n")
```

Output:

```
$ python display_sections.py olib.exe
UPX0 0x1000 0x1f000 0 UPX1
0x20000 0xe000 53760 .rsrc 0x2e000
0x6000 24576
```

Examining the compilation Timestamp

What is Compilation Timestamp?

The PE header of an executable contains a TimeDateStamp field that records when the binary was compiled. This information helps malware analysts estimate the age of the malware and understand when it may have been created or first deployed.

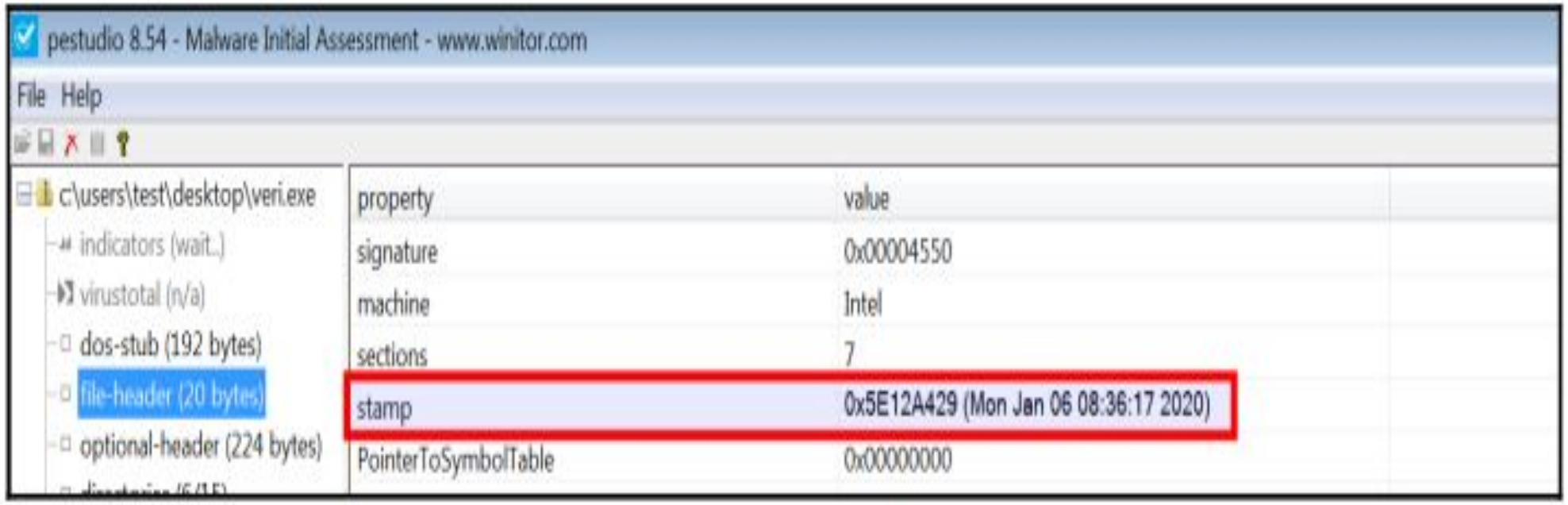
Why Compilation Timestamp is important in Malware Analysis??

The compilation timestamp is useful for building a timeline of malware campaigns and correlating samples. Analysts often compare timestamps across multiple samples to identify related malware versions or attack waves. However, attackers may intentionally modify this timestamp to hide the real compilation time.

Examining the compilation Timestamp

Detecting Anomalies in Timestamp

If a malware sample shows a **future timestamp** or an unrealistic date, it strongly suggests **timestamp manipulation**. Even though the real compilation time cannot be trusted, such anomalies themselves act as indicators of suspicious or malicious behavior.



Examining the compilation Timestamp

Extracting Compilation Timestamp using Python

The compilation timestamp can be extracted programmatically using Python's pefile module. The timestamp is stored as a Unix time value and can be converted into a human-readable format using the time module..

```
import pefile
import time

pe = pefile.PE("veri.exe")
timestamp = pe.FILE_HEADER.TimeDateStamp
print(time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(timestamp)))
```

Examining PE resources

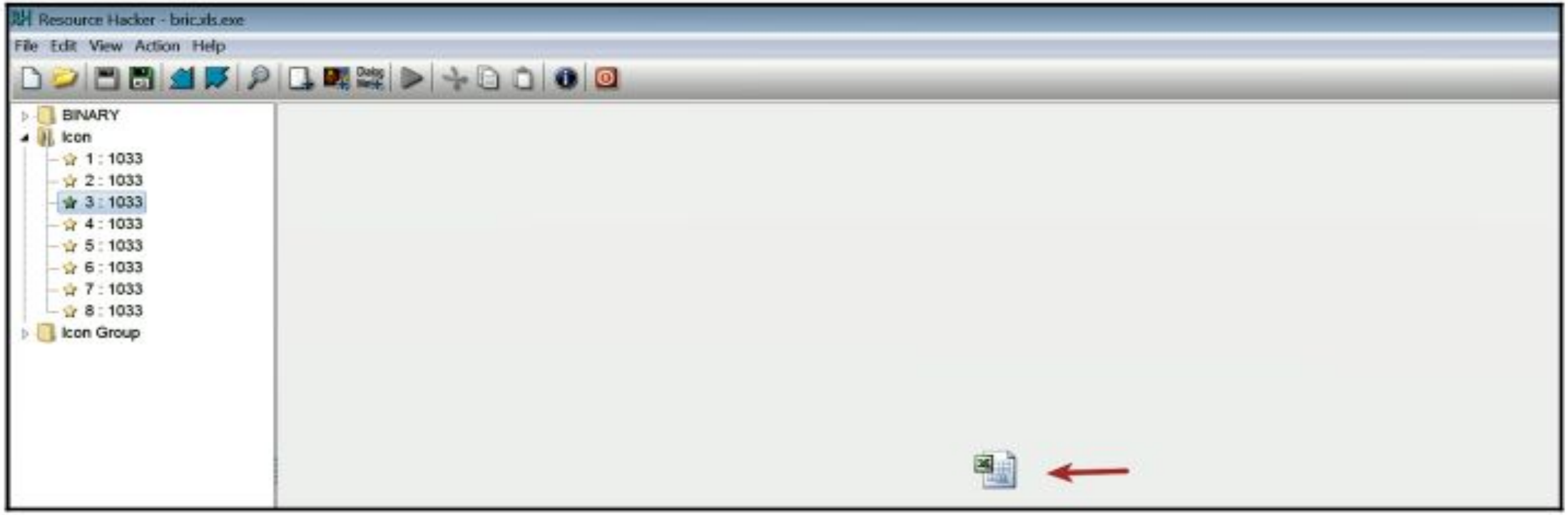
The resources required by the executable file such as icons, menu, dialog, and strings are stored in the resource section (.rsrc) of an executable file. Often, attackers store information such as additional binary, decoy documents, and configuration data in the resource section, so examining the resource can reveal valuable information about a binary.

The resource section also contains version information that can reveal information about the origin, company name, program author details, and copyright information

Resource Hacker (`http://www.angusj.com/resourcehacker/`) is a great tool to examine, view, and extract the resource from a suspect binary. Let's take an example of binary that looks like an Excel file on the disk (notice how the file extension is changed to `.xls.exe`), as shown here:

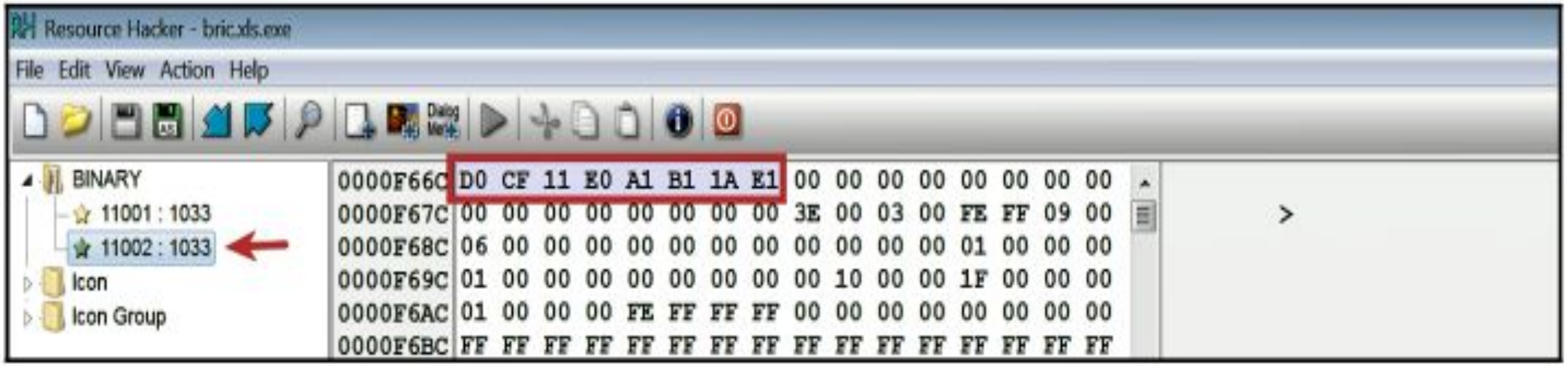
Examining PE resources

Loading a malicious binary in resource hacker shows three resources (*Icon*, *Binary*, and *Icon Group*). The malware specimen uses the icon of Microsoft Excel (to give the appearance of an excel sheet):



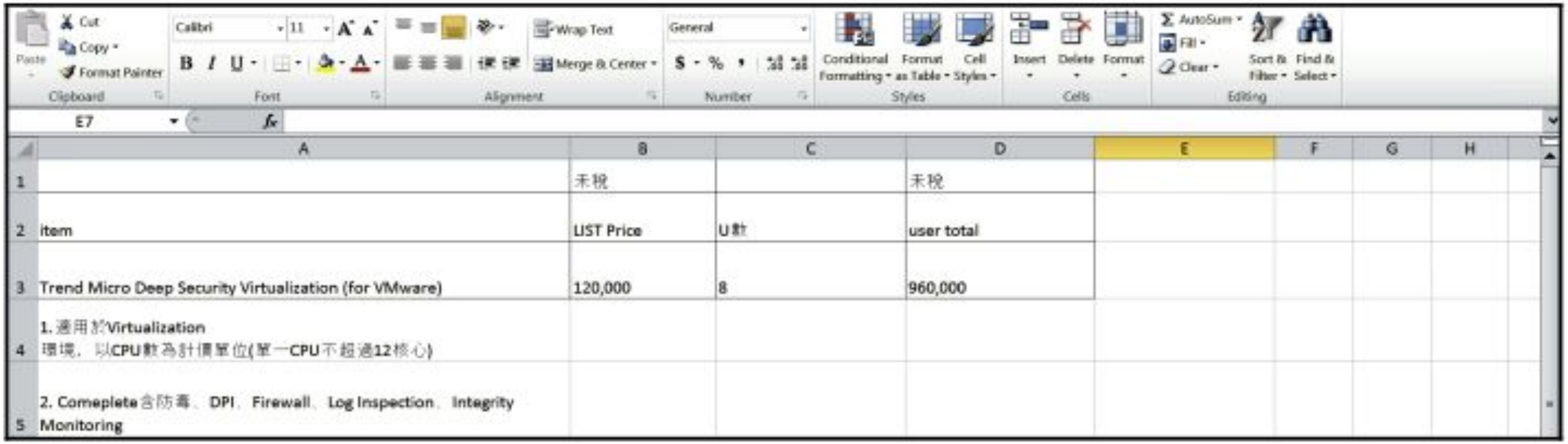
Examining PE resources

The executable also contains binary data; one of them has a file signature of `D0 CF 11 E0 A1 B1 1A E1`. This sequence of bytes represents the file signature for a Microsoft Office document file. The attackers, in this case, stored a decoy excel sheet in the resource section. Upon execution, the malware is executed in the background, and this decoy excel sheet is displayed to the user as a diversion:



Examining PE resources

To save the binary to disk, right-click on the resource that you want to extract and click on **Save Resource to a *.bin file**. In this case, the resource was saved as `sample.xls`. The following screenshot shows the decoy excel sheet that will be displayed to the user:



	A	B	C	D
1		未稅		未稅
2	item	LIST Price	U數	user total
3	Trend Micro Deep Security Virtualization (for VMware)	120,000	8	960,000
4	1. 適用於Virtualization 環境，以CPU數為計價單位(單一CPU不超過12核心)			
5	2. Comeplete含防毒、DPI、Firewall、Log Inspection、Integrity Monitoring			

Just by exploring the contents of the resource section, a lot can be learned about the malware characteristics.

Comparing and Classifying the Malware

- ✔ Comparing a suspicious malware sample with previously analyzed samples helps identify its **malware family**, behavior, and similarities with known malware.
- ✔ While cryptographic hashing (MD5/SHA1/SHA256) is a great technique to detect identical samples, it does not help in identifying similar samples.
- ✔ Very often, malware authors change minute aspects of malware, which changes the hash value completely

Comparing and Classifying the Malware

1. Classifying Malware using Fuzzy Hashing:

- ✔ Fuzzy hashing is a great method to compare files for similarity.
- ✔ ssdeep (http://ssdeep. sourceforge.net) is a useful tool to generate the fuzzy hash for a sample, and it also helps in determining percentage similarity between the samples.
- ✔ This technique is useful in comparing a suspect binary with the samples in a repository to identify the samples that are similar; this can help in identifying the samples that belong to the same malware family or the same actor group.

Comparing and Classifying the Malware

```
$ ssdeep veri.exe
ssdeep,1.1--blocksize:hash:hash,filename
49152:op398U/qCazcQ3iEZgcwwGF0iWC28pUtu6On2spPHlDB:op98USfcy8cwF2bC28pUtsRp
tDB,"/home/ubuntu/Desktop/veri.exe"
```

To demonstrate the use of fuzzy hashing, let's take an example of a directory consisting of three malware samples. In the following output, you can see that all three files have completely different MD5 hash values:

```
$ ls
aiggs.exe jnas.exe veri.exe

$ md5sum *
48c1d7c541b27757c16b9c2c8477182b aiggs.exe
92b91106c108ad2cc78a606a5970c0b0 jnas.exe
ce9ce9fc733792ec676164fc5b2622f2 veri.exe
```

The pretty matching mode (`-p` option) in ssdeep can be used to determine percentage similarity. From the following output, out of the three samples, two samples have 99% similarity, suggesting that these two samples probably belong to the same malware family:

```
$ ssdeep -pb *
aiggs.exe matches jnas.exe (99)
jnas.exe matches aiggs.exe (99)
```

Comparing and Classifying the malware

You might have a directory containing many malware samples. In that case, it is possible to run `ssdeep` on directories and subdirectories containing malware samples using the recursive mode (`-r`) as shown here:

```
$ ssdeep -lrpa samples/
samples//aiggs.exe matches samples//crop.exe (0)
samples//aiggs.exe matches samples//jnas.exe (99)

samples//crop.exe matches samples//aiggs.exe (0)
samples//crop.exe matches samples//jnas.exe (0)

samples//jnas.exe matches samples//aiggs.exe (99)
samples//jnas.exe matches samples//crop.exe (0)
```

You can also match a suspect binary with a list of file hashes. In the following example, the ssdeep hashes of all the binaries were redirected to a text file (`all_hashes.txt`), and then the suspect binary (`blab.exe`) is matched with all the hashes in the file. From the following output, it can be seen that the suspect binary (`blab.exe`) is identical to `jnas.exe` (100% match) and has 99% similarity with `aiggs.exe`. You can use this technique to compare any new file with the hashes of previously analyzed samples:

```
$ ssdeep * > all_hashes.txt
$ ssdeep -m all_hashes.txt blab.exe
/home/ubuntu/blab.exe matches all_hashes.txt:/home/ubuntu/aiggs.exe (99)
/home/ubuntu/blab.exe matches all_hashes.txt:/home/ubuntu/jnas.exe (100)
```

Comparing and Classifying the Malware

In Python, the fuzzy hash can be computed using *python-ssdeep* (`https://pypi.python.org/pypi/ssdeep/3.2`). The installation of the *python-ssdeep* module on Ubuntu Linux VM was covered in `Chapter 1`, *Introduction to Malware Analysis*. To calculate and compare fuzzy hashes, the following commands can be used in the script:

```
$ python
Python 2.7.12 (default, Nov 19 2016, 06:48:10)
>>> import ssdeep
>>> hash1 = ssdeep.hash_from_file('jnas.exe')
>>> print hash1
384:13gexUw/L+JrgUon5b9uSDMwE9Pfg6NgrWoBYi51mRvR6JZlbw8hqIusZzZXe:pIAKG91Dw
1hPRpcnud
>>> hash2 = ssdeep.hash_from_file('aiggs.exe')
>>> print hash2
384:13gexUw/L+JrgUon5b9uSDMwE9Pfg6NgrWoBYi51mRvR6JZlbw8hqIusZzZWe:pIAKG91Dw
1hPRpcnu+
>>> ssdeep.compare(hash1, hash2)
99
>>>
```

Comparing and Classifying the malware

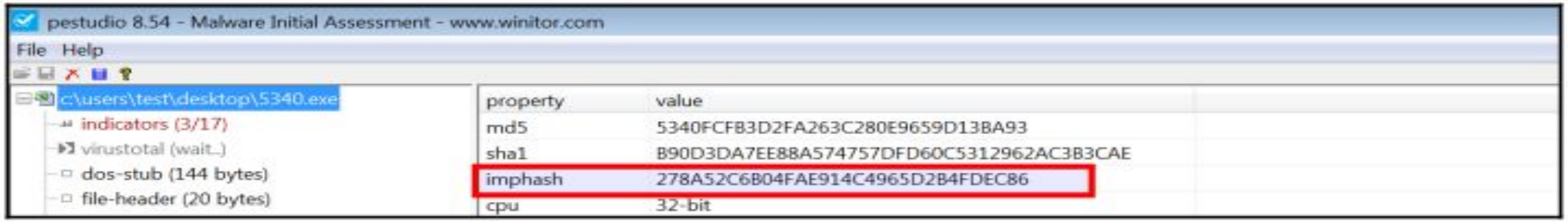
2. Classifying Malware using Import Hash

- ✔ Import Hashing is another technique that can be used to identify related samples and the samples used by the same threat actor groups.
- ✔ Import hash (or imphash) is a technique in which hash values are calculated based on the library/imported function (API) names and their particular order within the executable.
- ✔ If the files were compiled from the same source and in the same manner, those files would tend to have the same imphash value.
- ✔ During your malware investigation, if you come across samples that have the same imphash values, it means that they have the same import address table and are probably related.

Comparing and Classifying the malware

For detailed information on import hashing, and how it can be used to track threat actor groups, read `https://www.fireeye.com/blog/threat-research/2014/01/tracking-malware-import-hashing.html`.

When you load an executable into *pestudio*, it calculates the imphash as shown here:



Comparing and Classifying the malware

In Python, imphash can be generated using the *pefile* module. The following Python script takes the sample as input and calculates its imphash:

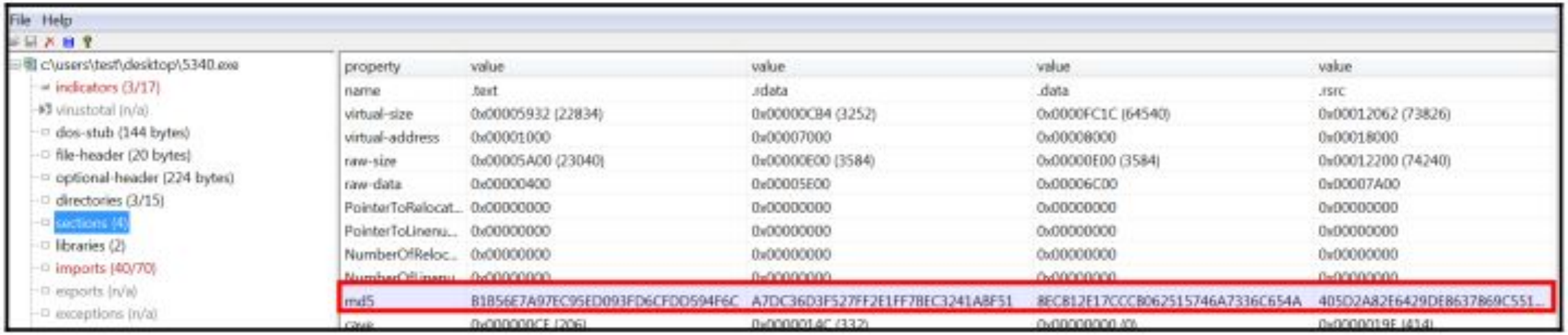
```
import pefile
import sys

pe = pefile.PE(sys.argv[1])
print pe.get_imphash()
```

The output as a result of running the preceding script against a malware sample is shown here:

```
$ python get_imphash.py 5340.exe
278a52c6b04fae914c4965d2b4fdec86
```

7.3 Classifying Malware Using Section Hash

Similar to import hashing, section hashing can also help in identifying related samples. When an executable is loaded in *pestudio*, it calculates the *MD5* of each section (`.text`, `.data`, `.rdata`, and so on.). To view the section hashes, click on **sections** as shown here:



Comparing and Classifying the malware

In Python, pefile module can be used to determine the section hashes as shown here:

```
>>> import pefile
>>> pe = pefile.PE("5340.exe")
>>> for section in pe.sections:
...     print "%s\t%s" % (section.Name, section.get_hash_md5())
...
.text b1b56e7a97ec95ed093fd6cfdd594f6c

.rdata a7dc36d3f527ff2e1ff7bec3241abf51
.data 8ec812e17cccb062515746a7336c654a
.rsrc 405d2a82e6429de8637869c5514b489c
```

7.4 Classifying Malware using Yara

Comparing and Classifying the malware

Classifying Malware using Yara

A malware sample can contain many strings or binary indicators; recognizing the strings or binary data that are unique to a malware sample or a malware family can help in malware classification.

YARA (http://virustotal.github.io/yara/) is a powerful malware identification and classification tool. Malware researchers can create YARA rules based on textual or binary information contained within the malware specimen. These YARA rules consist of a set of strings and a Boolean expression, which determines its logic. Once the rule is written, you can use those rules to scan files using the YARA utility or you can use yara-python to integrate with your tools

Comparing and Classifying the malware

Classifying Malware using Yara

Installing Yara

In Kali Linux VM

- **Update and Upgrade**

sudo apt update && sudo apt upgrade -y

Note: If already updated and upgraded this cmd can be ignored.

- **Install yara**

sudo apt install yara -y

- **yara version**

yara --version

Comparing and Classifying the malware

Yara Rule Basics:

In Kali Linux VM—Save the file as rules.yar

```
rule suspicious_strings
{
strings:
    $a = "Synflooding"
    $b = "Portscanner"
    $c = "Keylogger"

condition:
    ($a or $b or $c)
}
```

There are 3 sections:

- ✔ Rule identifier
- ✔ Strings definition
- ✔ Condition section

Comparing and Classifying the malware

Running Yara

In Kali Linux VM

yara windows_malware_generic.yar suspicious.bin

```
rule suspicious_strings
{
strings:
    $mz = {4D 5A}
    $a = "Synflooding" ascii wide nocase
    $b = "Portscanner" ascii wide nocase
    $c = "Keylogger" ascii wide nocase
condition:
    ($mz at 0) and ($a or $b or $c)
}
```

Now, running the preceding rule only detected the executable files:

```
$ yara -r suspicious.yara samples/
suspicious_strings samples//spybot.exe
suspicious_strings samples//wuamqr.exe
```

Comparing and Classifying the malware

Applications of Yara

YARA is a powerful tool; creating YARA rules to scan a repository of known samples can identify and classify samples having the same characteristics. The strings that you use in the rule might create false positives. It is a good idea to test your signatures against known good files and also to think of situations that might trigger false positives.

Applications of Yara:

1. If malware embeds a decoy Microsoft Office document (like an Excel file) inside the PE executable's resource section. The YARA rule detects this by searching for the Office file signature (hex pattern such as OLE header) at an offset greater than 1024 bytes, which skips the normal PE header and checks deeper inside the file for embedded content.

```
rule embedded_office_document
{
meta:
description = "Detects embedded office document"

strings:
    $mz = { 4D 5A }
    $a = { D0 CF 11 E0 A1 B1 1A E1 }
condition:
    ($mz at 0) and $a in (1024..filesize)
}
```

Running the preceding yara rule detected only the sample that contained the embedded excel document:

```
$ yara -r embedded_doc.yara samples/
embedded_office_document samples//5340.exe
```

Applications of Yara

2. The following example detects a malware sample called 9002 RAT using the serial number of its digital certificate. RAT 9002 used a digital certificate with a serial number 45 6E 96 7A 81 5A A5 CB B9 9F B8 6A CA 8F 7F 69 (https://blog.cylance.com/another9002-trojan-variant). The serial number can be used as a signature to detect samples that have the same digital certificate:

```
rule mal_digital_cert_9002_rat
{
meta:
    description = "Detects malicious digital certificates used by RAT 9002"
    ref = "http://blog.cylance.com/another-9002-trojan-variant"

strings:
    $mz = { 4D 5A }
    $a = { 45 6e 96 7a 81 5a a5 cb b9 9f b8 6a ca 8f 7f 69 }

condition:
    ($mz at 0) and ($a in (1024..filesize))
}
```

Running the rule detected all samples with the same digital certificate, and all of these samples turned out to be *RAT 9002* samples:

```
$ yara -r digi_cert_9002.yara samples/
mal_digital_cert_9002_rat samples//ry.dll
mal_digital_cert_9002_rat samples//rat9002/Mshype.dll
mal_digital_cert_9002_rat samples//rat9002/bmp1f.exe
```

Applications of Yara

YARA rule can also be used to detect packers. Exeinfo PE uses signatures stored in a plain text file called userdb.txt. The following is an example signature format used by Exeinfo PE to detect the UPX packer:

```
[UPX 2.90 (LZMA)]
signature = 60 BE ?? ?? ?? ?? 8D BE ?? ?? ?? ?? 57 83 CD FF EB 10 90 90 90
90 90 90 8A 06 46 88 07 47 01 DB 75 07 8B 1E 83 EE FC 11 DB 72 ED B8 01 00
00 00 01 DB 75 07 8B 1E 83 EE FC 11 DB 11 C0 01 DB
ep_only = true
```

The ep_only=true in the preceding signature means that Exeinfo PE should only check for the signature at the program's address of entry point (which is where the code starts executing). The preceding signature can be converted to a YARA rule.

Applications of Yara

```
import "pe"
rule UPX_290_LZMA
{
meta:
    description = "Detects UPX packer 2.90"
    ref = "userdb.txt file from the Exeinfo PE"

strings:
    $a = { 60 BE ?? ?? ?? ?? 8D BE ?? ?? ?? ?? 57 83 CD FF EB 10 90 90 90
90 90 90 8A 06 46 88 07 47 01 DB 75 07 8B 1E 83 EE FC 11 DB 72 ED B8 01 00
00 00 01 DB 75 07 8B 1E 83 EE FC 11 DB 11 C0 01 DB }

condition:
    $a at pe.entry_point
}
```